

SPD well-ordering in KP with an uncountable ordinal

DEFINITIONS & PROOFS / ENGLISH EDITION

2026-09-14. Paper proof manuscript for the fixed Slot Profile Diagrams (SPD) definition. This is not a Lean certificate: no SPD well-ordering theorem has been formalized in this repository, and the KP derivation below has not been internally encoded in Lean. The finite reference programs and their tests do not change that status.

Work in

$$S = KP_\omega + \exists \nu (\nu > \omega \text{ is an ordinal and no set function } \omega \rightarrow \nu \text{ exists}).$$

KP includes full set induction, literal Δ_0 -Separation and Δ_0 -Collection. The proof uses no power set, full separation, general choice, regularity assumption on the uncountable ordinal, or additional large cardinal. The existing IPD paper supplies explicitly identified KP constructions, not an assumption that SPD is well-ordered.

1. Statement and fixed scope

Use the SPD definition, including all four features: LATENT slots, the highest visible formal-head fiber, the canonical strict connector guard, and typed source carry only after the first block. A notation is a finite list of finite columns of integer quadruples. Derived heads are temporary semantics, not additional notation input.

Let 0 be the empty graph. Let S_n have $n + 1$ columns: column 0 is empty, and column $j > 0$ is the singleton $(j - 1, j - 1, j - 1, \text{SELF})$. In particular, S_0 is one empty column, not 0. Let Std be the finite graphs reachable from some S_n by finitely many fundamental-sequence operations. The optional symbol Top is greater than every finite standard graph and satisfies $\text{Top}[n] = S_n$.

Paper theorem. In S :

1. Fundamental-sequence steps from nonempty structurally valid canonical finite graphs are well-founded.
2. The column lexicographic order on Std is a well-order; adding Top preserves well-ordering.

The first assertion is about the expansion relation on *all raw graphs*, not their global column order. Raw column order is not well-founded. The second assertion therefore requires the separate standard-domain argument in section 9.

This manuscript does not claim an order-type identification or a comparison with the whole IPD, ARD, or wY notation systems. It does not include experimental refresh, splice, or alternative merge rules.

2. Finite syntax and order interfaces

2.1 Slots and derived heads

In column j , a row (h, p, s, q) satisfies $h, s \leq p < j$. The root slot is either ROOT $q \leq p$, SELF $q = j$, or LATENT $q = j + 1$. LATENT is a tag, not a pointer to the next column. At each fixed (h, p, s) retain only the largest q .

For an ordered alphabet X , let $U(X)$ consist of constants Z and $x \in X$, and finite terms $N(h; t_0, \dots, t_{r-1})$ with $h, t_i \in U(X)$. Constants have order $Z < X$ and lie below all compound terms. Compare compounds by

ordinary-child dominance and then the lexicographic root key $(h, r, t_0, \dots, t_{r-1})$, as in the definition. In particular, a child that is at least the opposite term determines the comparison before the root keys are consulted. The recursively formed head h is *not* an additional ordinary-child dominance branch. There is no global CAP.

Derived templates H_j use the formal alphabet

$$Z < \text{ROOT}(0) < \text{ROOT}(1) < \dots < A < B.$$

Ignore LATENT rows. If no visible row remains, put $H_j = Z$. Otherwise select the unique largest key (H_h, h) in this common formal alphabet, and call its address h_* . Read just the rows with $h = h_*$, in decreasing address order (p, s, q) . A row contributes the two ordinary arguments H_s, Q , where

$$Q = \begin{cases} A & q = p, \\ B & q = j, \\ \text{ROOT}(q) & q < p. \end{cases}$$

The head of the resulting N is H_{h_*} . Recursively referenced templates retain the same two holes A, B . Finite induction gives

$$\text{literalROOT}(H_j) \subseteq \{0, \dots, j-1\}. \quad (2.1)$$

The interpretation of H_i at a future parent $p \geq i$ sets $A = p$ and uses one common largest SELF marker for B . Each row's profile is

$$P(h, p, s, q) = (H_h[A = p], h, H_s[A = p], s, q). \quad (2.2)$$

Comparisons across different parents instantiate A at those different parents. Columns store rows in decreasing (p, P) order; the controller is the largest (P, p) instead. These two orders must not be interchanged. Graph comparison is lexicographic by columns, with a proper prefix smaller.

2.2 The finite properties used later

The fixed algorithm has the following elementary properties:

$$G[0] = G \text{ with its last column deleted}, \quad G[n] \preceq_{\text{prefix}} G[n+1]. \quad (2.3)$$

An empty last column is deleted for every index. If it is nonempty, a positive index produces successive blocks, each consisting of a lowered seam, an optional nonempty connector, and a complete pure source copy. The old prefix before the last column is unchanged. Block b depends on its preceding blocks, not on the final number of blocks, proving the second part of (2.3).

In the first seam, the controller root lowers through

$$\text{LATENT} \mapsto \text{SELF} \mapsto p \mapsto p-1 \mapsto \dots \mapsto 0 \mapsto \text{deletion}.$$

All inserted rows at this parent have a strictly smaller four-coordinate pair $(H_h[A = p], h, H_s[A = p], s)$. Noncontroller rows remain. The controller is maximal among rows of its own parent, and every added row is below it even when its root is LATENT. Thus the first changed row in that parent's decreasing list strictly decreases or disappears; all other parent groups are unchanged. The first block has no carry. It follows that

$$G \neq 0 \implies G[n] <_{\text{col}} G \text{ for every } n. \quad (2.4)$$

All references in every block are backward and satisfy the stated support bounds. Normalization only removes rows. These facts prove closure and strict column descent by finite induction, without any transfinite assumption. The convention $0[n] = 0$ is not included among strict expansion steps.

3. Actual ordinal ranks for the semantic profiles

3.1 The recursive-head order

Put $d(Z) = d(x) = 0$ and

$$d(N(h; \vec{t})) = \max(d(h) + 1, \max_i d(t_i)).$$

Finite induction on the combined sizes of two terms shows

$$d(u) < d(v) \implies u < v. \quad (3.1)$$

Indeed, no ordinary child of u can dominate v . If an ordinary child of v has depth $d(v)$, the induction hypothesis makes it larger than u . Otherwise v gets its depth from its head, whose depth is strictly greater than that of u 's head, and the head comparison decides. Constants give the base case. This also shows that a head is smaller than its compound, although it is not an extra comparison branch.

Consequently $U_{\leq n}(X)$ is an initial segment of $U(X)$. For $n > 0$, it is the finite-term LPO algebra whose ranked symbols are constants and pairs (h, r) with $h \in U_{\leq n-1}(X)$ and fixed arity r . Ordinary children remain in $U_{\leq n}(X)$. The order on symbols is the already ranked lexicographic order on (h, r) , with constants below compounds.

Apply IPD Appendix A, Theorem A and its finite-layer construction at each n . The relevant conclusion is an *actual set ordinal rank* for this ground LPO algebra, not merely the absence of a descending sequence. The construction uses canonical collapses of individual closed initial segments, compatible predecessor ranks, and collection of finite layer histories; it does not invoke a general collapse theorem for arbitrary internal well-orders. Since the depth layers here are initial segments, their canonical initial-segment ranks agree on overlaps. Collecting the finite histories and their compatible union gives an actual strict ordinal rank for $U(X)$ in KP.

This is the only tree-order result required from the IPD paper. It is applied to finite *derived* semantics; it does not introduce trees into SPD input.

3.2 Two endpoint letters

Fix an uncountable set ordinal κ . Use the explicitly ranked alphabet

$$X_\kappa = (\kappa + 1) \times \{0, 1\}$$

in lexicographic order, writing $\alpha^0 = (\alpha, 0)$ and $\alpha^1 = (\alpha, 1)$. Under a strictly increasing label vector f , ROOT i is $f(i)^0$, the parent hole is a^0 , SELF is b^0 , and LATENT is b^1 . Both endpoint letters remain constants below every compound term. The right endpoint of the recursive relation is the *base ordinal* b , not b^1 .

The profile order is the fixed lexicographic product

$$\mathcal{Q}_\kappa = U(X_\kappa) \times X_\kappa \times U(X_\kappa) \times X_\kappa \times X_\kappa.$$

Finite ordinal products and the preceding rank give an actual set ordinal Ξ and a strict rank $\rho : \mathcal{Q}_\kappa \rightarrow \Xi$. This construction precedes the demand relation and uses no fact about SPD expansion.

3.3 Naturality and positive templates

Strict alphabet embeddings preserve and reflect U comparison, by induction on the finite comparison computation. They therefore preserve profile comparison. All N constructors are monotone in their head and ordinary arguments, by the same LPO context argument. Fixed lexicographic products are monotone under coordinatewise increase.

For a fixed graph, the active fiber in H_j is unchanged under any legal evaluation with parent $a \geq f(j)$ and endpoint $b > a$: (2.1) puts all literal ROOTs below a , so the formal order $\text{ROOT} < A < B$ is mapped strictly into $\text{ROOT} < a^0 < b^0$. Address tie-breaks and the decreasing address order are preserved as well. Once the graph is fixed, H_j is thus a fixed finite positive expression, not a comparison-dependent choice that changes as a varies.

A positive future-profile template is a finite expression built from Z , the label variables in a fixed prefix u , a new parent variable z , the endpoint letters b^0, b^1 , the N constructors, and the fixed profile tuple. Its support must make it a valid profile at parent z whenever

$$\omega < u_0 < \cdots < u_{m-1} < z < b.$$

No old parent value is allowed as an extra fixed ROOT outside u . Empty prefixes and empty template families are allowed. For these templates,

$$\max(\{\omega\} \cup \text{ran } u) < d \leq a < b \implies U(u, d, b) \leq U(u, a, b). \quad (3.2)$$

The connector templates in section 8 have exactly this form. This is monotonicity under relabelling a *fixed graph*. Adding or removing rows may change the active fiber; no monotonicity under such graph changes is asserted.

4. A bounded finite-demand relation with new parents

4.1 Representation and finite requests

Write $\text{Inc}_b(f)$ for a finite strictly increasing vector with all labels strictly between ω and b . A profile is valid at (a, b) if $\omega < a < b \leq \kappa$ and all its letters are ROOT letters r^0 with $r \leq a$ or the endpoint letters b^0, b^1 . We may use this larger semantic domain even though actual SPD profiles have additional syntactic restrictions.

Relative to a relation R^+ , define $\text{Rep}(G, f)$ by requiring $R^+(P_e[f], f(p), f(j))$ for every row e with parent p in column j , together with increasing labels. The derived heads are those of the fixed graph G .

A finite endpoint request N is a list of pairs (p, τ) : p is a prefix address, and τ is a finite profile template whose ROOT addresses are at most p . Put

$$\text{End}_b(N, f) \iff \bigwedge_{(p, \tau) \in N} R^+(\tau[f, b], f(p), b).$$

It is admissible for (T, a, b) when every requested pair satisfies

$$(\tau[f, b], f(p)) <_{\text{lex}} (T, a). \quad (4.1)$$

Endpoint templates may name deeply nested SELF; changing the endpoint replaces all its occurrences, and replaces LATENT's second endpoint letter at the same time.

4.2 The two defining clauses

Define $R^+(T, a, b)$ to be the validity guard and the conjunction of the following clauses.

FR (finite reflection). For every finite G, N, c, f satisfying

$$\text{Rep}(G, f), \quad \text{Inc}_b(f), \quad f(c) = a, \quad \text{End}_b(N, f), \quad (4.1),$$

there is g such that

$$\text{Rep}(G, g), \quad \text{Inc}_a(g), \quad g \upharpoonright c = f \upharpoonright c, \quad \text{End}_a(N, g). \quad (\text{FR})$$

NP (new parent). For every finite graph K of length $m + 1$, prefix vector u of length m , and finite family of positive future-profile templates U_i , if

$$\text{Rep}(K, u \frown [a]), \quad U_i(u, a, b) < T \quad \text{for every } i,$$

then there is d with $\max(\{\omega\} \cup \text{ran } u) < d < a$ such that

$$\begin{aligned} &\text{Rep}(K, u \frown [d]), \\ &U_i(u, d, b) < T, \quad R^+(U_i(u, d, b), d, b) \quad \text{for every } i. \end{aligned} \quad (\text{NP})$$

NP keeps the whole prefix u fixed. It supplies lower profiles at a genuinely new parent, not arbitrary cross-parent monotonicity.

4.3 Why this is a definition, not a reflection axiom

Recurse on $(b, \rho(T), a)$, using the ordinal stage code

$$((\kappa + 1) \cdot \Xi) \cdot b + (\kappa + 1) \cdot \rho(T) + a.$$

In FR, internal and output queries have smaller right endpoint; input endpoint queries have the same endpoint but smaller (profile, parent). In NP, input representation queries have endpoint at most $a < b$, and output representation queries have endpoint at most $d < a$. The future queries still end at b , but the guard $U_i(u, d, b) < T$ is checked *before* looking them up, so their profile ranks are smaller.

Finite graphs, templates and evaluation records have set-sized finite-code sources. All label vectors lie in the existing set $(\kappa + 1)^{<\omega}$, and every candidate d lies in κ . With a fixed earlier history, every quantifier in the clauses is bounded. The compatible-history construction of IPD section 4.3 therefore produces the actual set R^+ : successor rows use bounded separation, and limit histories are uniquely compatible and collected by Σ_1 -Collection. No recursion over the unknown standard order occurs.

Same-parent weakening. If $T' \leq T$ is valid at (a, b) and $R^+(T, a, b)$, then $R^+(T', a, b)$. FR has fewer admissible requests. For NP, an input below T' is below T ; use NP for T , then (3.2) gives $U_i(u, d, b) \leq U_i(u, a, b) < T'$. This proves the output guard for T' . It does not prove $R^+(T, d, b)$ from $R^+(T, a, b)$.

5. Countable witness closure

Temporarily work in $S + V = L$. Take the least uncountable ordinal κ . IPD section 4.1 constructs a set function $e : \kappa \times \omega \rightarrow \kappa$ uniformly enumerating every nonzero ordinal below κ , by collection and bounded least selection in L . It does not assume κ is regular.

Only after constructing the full set R^+ do we define the following finite-arity operations. Finite shapes have natural codes. Order input packets first by shape code and then by their fixed-length label vectors

lexicographically; the ordinal sum of the corresponding finite powers of κ is an explicit rank. Thus every nonempty packet set has a least member. This is the selection construction in IPD section 12.1.

- $\text{Bad}_{\tau,i}(\vec{r}, a)$ returns coordinate i of the least admissible input witnessing failure of the **FR clause** for $R^+(\tau[\vec{r}, \kappa], a, \kappa)$. It returns 0 when no such input exists. It is not a selector for arbitrary failure of NP.
- $\text{Ext}_{G,N,c,i}(u)$ returns coordinate i of the least representation of G below κ that keeps the prefix u and satisfies $\text{End}_\kappa(N, -)$. It fixes neither a new cut label nor a controller nor an admissibility condition.
- $\text{LastExt}_{K,\vec{U}}(u)$ is the least $z < \kappa$ for which $u \cap [z]$ represents K and every $R^+(U_i(u, z, \kappa), z, \kappa)$ holds. It returns 0 if none exists. Neither the old last label a , the controller T , nor a condition $U_i < T$ is a fixed parameter or constraint of this operation.

Their graphs are bounded set definitions from R^+ and the finite-code and label sources. There are countably many operation symbols; ordinal ROOT values are finite arguments, not an uncountable set of symbols.

Close 0, ω , γ under these operations and $\alpha \mapsto e(\alpha, n)$ for each $n < \omega$, by evaluating finite operation terms. As in IPD section 12.2, the range H is an actual enumerable set. Closure under e makes it downward closed, hence an ordinal $\theta \leq \kappa$. Its enumeration excludes $H = \kappa$, so

$$\omega, \gamma < \theta < \kappa.$$

Call such an ordinal closed. Closed ordinals occur above every $\gamma < \kappa$. No countable sequence of independently chosen witnesses and no regularity argument is needed.

6. Endpoint agreement and closed-parent supply

6.1 Endpoint agreement

If θ is closed and all ROOT parameters are at most $a < \theta$, then

$$R^+(\tau[\vec{r}, \kappa^0, \kappa^1], a, \kappa) \iff R^+(\tau[\vec{r}, \theta^0, \theta^1], a, \theta). \quad (\text{EA})$$

Induct on $(\rho(\tau[\vec{r}, \theta^0, \theta^1]), a)$. Replacing the two highest endpoint letters is a strict alphabet embedding, so all admissibility and strict-profile comparisons agree.

For FR from the κ side to the θ side, an input below θ has the same internal representation. Each of its endpoint requests is lower in the induction order and can be transferred to κ . FR then gives an output below a , unchanged between the two endpoints. For the reverse direction, if FR fails at κ , the least Bad packet has all its labels below θ , because its ROOT parameters and a are below θ . The induction hypothesis transfers its lower endpoint requests to θ . An FR output there would be the same output at κ , a contradiction.

For NP, every input prefix u and every possible output d is already below $a < \theta$. Their internal representation queries do not change. Its future queries satisfy $U_i < T$ before being read, so the induction hypothesis transfers each one. Naturality transfers both the input and output strict guards. Thus each fixed NP input has exactly the same feasible d at the two endpoints. No Bad selector for NP is required.

This proves the two defining clauses separately and hence proves (EA). It does not use the supply theorem below.

6.2 Supply at a closed parent

For every closed θ ,

$$T \text{ valid at } (\theta, \kappa) \implies R^+(T, \theta, \kappa). \quad (\text{CS})$$

Prove this by induction on the actual rank of T .

For an FR input with cut label θ , the old representation witnesses existence for $\text{Ext}(u)$, where u is the prefix before the cut. Every coordinate of u is below θ . Closure supplies the selected whole representation below θ , retaining u and satisfying End_κ . Equation (EA) changes these requests to End_θ . Ext did not retain the old cut or controller, so this compression is not obstructed by $\text{ROOT} = \theta$ in T .

For an NP input, each $U_i(u, \theta, \kappa) < T$ is valid at parent θ . The profile induction hypothesis gives all its R^+ facts. Hence $z = \theta$ witnesses existence for $\text{LastExt}_{K, \vec{U}}(u)$. Its only ordinal arguments are $u < \theta$; closure supplies a selected $d < \theta$, preserving the representation and the future facts. Positivity gives

$$U_i(u, d, \kappa) \leq U_i(u, \theta, \kappa) < T,$$

which is precisely the remaining NP guard. This is induction on strictly smaller profiles, not an assumption of the desired supply.

6.3 Every finite raw graph has a representation

For closed $\alpha < \beta$, (CS) at α followed by (EA) at β supplies every valid profile at (α, β) . Given a finite graph, choose a finite increasing sequence of closed heights above ω and assign them to its columns. Equation (2.1) guarantees validity for every row. Therefore every structurally valid finite graph has a representation. Subsequent compressed labels need not themselves be closed: their required facts are supplied by FR and NP.

7. Exact source transport and typed carry

A pure source copy uses a strictly increasing address map ϕ , fixes old addresses below the cut, and sends the copied source columns to their new locations. Decode ROOT, SELF and LATENT at the original child before moving a row. Thus

$$\text{SELF}_j \mapsto \text{SELF}_{\phi(j)}, \quad \text{LATENT}_j \mapsto \text{LATENT}_{\phi(j)},$$

the latter having raw code $\phi(j) + 1$, not $\phi(j + 1)$.

Finite induction on source columns proves the exact template identity

$$H'_{\phi(j)} = \phi_{\text{ROOT}}(H_j). \tag{7.1}$$

The map preserves the active formal-head key and its address tie-break, the visible/latent distinction, the three cases $q < p, q = p, q = j$, and the address ordering of the active fiber. Earlier references go either into the fixed prefix or into earlier pure copies. Under labels that restore the old source labels, (7.1) therefore restores every source row's full profile, parent and endpoint. Intervening seams and connectors do not enter the copied dependencies.

For typed carry, consider the original cut column C_c and a later block whose virtual cut is c_b . The first FR call fixes every address below c_b . Every reference of the transported cut column, including its literal ROOT support, is below c_b . Put its selected carried rows at a new seam with the old cut label a . Retagging their SELF/LATENT to the seam changes neither semantic endpoint letter. Thus their full R^+ facts are identical to the source facts.

The fixed rule selects all source LATENT rows and visible source rows whose head address equals the original controller's head address; any such subset is valid by the preceding argument. Normalization retains one of the already valid rows at each (h, p, s) , so it preserves representation. It may change the new seam's H or active fiber, which is harmless: section 8 uses the template derived from the actual normalized seam. Carry is absent

in block 0, preserving the strict first difference in (2.4). The full source copy is never filtered by the typed-carry test.

8. One complete block lowers the represented endpoint

Fix a representation F of an input with nonempty last column x , controller (h, c, s, q) , last label $\beta = F(x)$, cut label $a = F(c)$, and full controller profile T . Then $R^+(T, a, \beta)$ holds.

At the start of each block let ϕ be the current pure-source map and D the actual prefix before the virtual last column. Its labels f preserve the old labels on the source image: $f(\phi(i)) = F(i)$. Put $c_b = \phi(c)$, $h_b = \phi(h)$ and $s_b = \phi(s)$. The virtual controller at endpoint β still has exactly the old profile T . The invariant is initially the input prefix and will be restored by the last pure copy of the block.

8.1 First FR: the seam

The lower seam's endpoint requests at β consist of transported noncontroller rows, the lowered controller if present, and all inserted lower-pair LATENT rows at parent c_b .

Noncontroller requests are admissible because the controller maximizes (P, p) . Root lowering makes the controller profile smaller at the same parent. Every lower-pair row has profile strictly below T regardless of its final LATENT coordinate, so same-parent weakening supplies it. The finite root predecessor is taken in the current address order; even if it is a newly inserted address, it is still smaller than the old root and at most the parent.

FR gives a representation g of D , entirely below a , fixes indices below c_b , and gives the lower seam's End_a facts. Put the seam S at label a . In blocks $b \geq 1$, add typed carry using section 7. Normalize. We obtain

$$K = D + S, \quad \text{Rep}(K, g^\wedge[a]). \quad (8.1)$$

8.2 Why the canonical finite guard is exact

Freeze the old templates H_{h_b}, H_{s_b} . All their literal ROOT addresses are strictly below c_b , even when $h_b = c_b$ or $s_b = c_b$. Their values are therefore fixed by the first FR on this support.

In the finite alphabet ROOT $0 < \dots < \text{ROOT}(S-1) < S < B < C$, form

$$T^{\uparrow S} = (H_{h_b}[A = S, B], h_b^\uparrow, H_{s_b}[A = S, B], s_b^\uparrow, q^\uparrow), \quad (8.2)$$

where $i^\uparrow = S$ if $i = c_b$ and $i^\uparrow = i$ otherwise. ROOT $q = c_b$ is raised to S ; smaller ROOTs stay fixed; old SELF and LATENT become B and C . Crucially, if $h_b = c_b$, the first entry is still the *old* H_{c_b} , not the new H_S . The same caution applies to s_b .

For $k, t \leq S$, the candidate connector profile is

$$U_{k,t}^S = (H_k[A = S, B], k, H_t[A = S, B], t, C). \quad (8.3)$$

The implementation compares only the first four entries. This is exactly $U_{k,t}^S < T^{\uparrow S}$, because every candidate ends in the greatest root letter C and $q^\uparrow \leq C$: equality of the first four entries can never yield a strictly smaller full profile.

Interpret addresses $i < S$ as $g(i)^0$, S as a^0 , and B, C as β^0, β^1 . This is a strict alphabet embedding. It takes (8.3) to a valid positive future profile $U_{k,t}(g, a, \beta)$. It takes (8.2) *exactly* to T : literal ROOTs below c_b were fixed; the

raised cut coordinates restore $F(c) = a$; both holes and endpoint tags restore their old values. Thus

$$U_{k,t}^S < T^{\uparrow S} \iff U_{k,t}(g, a, \beta) < T. \quad (8.4)$$

This includes $k = S$ or $t = S$, changed active fibers, and newly present literal ROOTs in H_S . Those ROOTs are in g , below the new parent. All future templates are fixed expressions from the actual K , so (3.2) applies when the parent is lowered. No comparison is made in an illegal labelling or by rebinding an old head reference to H_S .

8.3 NP and the connector's two FR calls

Apply NP to (8.1) and the finite family admitted by (8.4). It returns $d < a$, keeps the whole vector g , represents K at $g \frown [d]$, and supplies

$$R^+(U_{k,t}(g, d, \beta), d, \beta), \quad U_{k,t}(g, d, \beta) < T. \quad (8.5)$$

If there are no connector rows, omit the connector; NP with the empty family already puts S below a .

Otherwise K now has no column labelled a , so FR with cut label a cannot be applied to K alone. For the proof only, append an empty column Q at label a . Use its cut and the connector rows as End_β requests. Equation (8.5) gives the facts and admissibility. FR keeps all labels of K fixed and gives End_a for the connector. Discard the auxiliary empty Q .

No actual parent, head, argument, or literal ROOT refers to Q . Endpoint tags are template symbols, not ROOT references to it. After deleting Q , retagging SELF/LATENT to the actual connector index preserves these requests exactly. Put the connector E at label a . A final FR call, with cut E and empty endpoint request, keeps K fixed and puts E at some $e < a$. In the nonempty case the resulting labels satisfy

$$\text{all labels of } D < d = \text{label}(S) < e = \text{label}(E) < a.$$

There are four demand calls: FR, NP, FR with the proof-only empty cut, and FR with empty requests. The helper column is not part of SPD output.

8.4 Restore the source and finish the block induction

Append the complete pure copy of the original columns $C_c, \dots, C_{x-1}$, assigning them their old labels $F(c), \dots, F(x-1)$. These begin at a , above every new seam/connector label. Their references below c are in the fixed prefix; all other source references are in the pure copy. Section 7 restores their exact R^+ facts and derived templates. This establishes the next block's source invariant.

There is at least one copied source column because $c < x$. Every completed block therefore ends at the old penultimate label

$$F(x-1) < F(x) = \beta. \quad (8.6)$$

Finite induction constructs a representation of every positive fundamental term with last label below β . If $n = 0$, or the old last column is empty, restriction of the old representation proves the same assertion, with the empty graph handled separately. Compression changes semantic labels, not the already written columns; the syntactic old-prefix condition remains exact.

9. From raw expansion ranks to the standard column well-order

9.1 A single ordinal-valued rank on all raw graphs

Structurally valid canonical finite graphs form a set $\mathcal{E} \subseteq \omega$ of finite codes. Finite computation records also give set-coded expansion and finite reachability relations. Section 6.3 supplies a representation of each member.

Define

$$\mu(0) = 0, \quad \mu(G) = \min\{f(|G| - 1) : f \in (\kappa + 1)^{<\omega}, \text{Rep}(G, f)\} \quad (G \neq 0).$$

The minimization is over a nonempty bounded set of ordinals. Its whole graph is obtained by bounded separation from $\mathcal{E} \times \kappa$, using the already constructed R^+ and finite evaluation sets. Thus μ is an actual set function, not a choice of representations or a class-valued informal rank.

For one fixed G , take any representation attaining its least last label; this local existential use requires no choice function. Section 8 gives a representation of every $G[n]$ with smaller last label. Consequently

$$G \neq 0 \implies \mu(G[n]) < \mu(G). \tag{9.1}$$

This proves well-foundedness of all raw expansion steps. It says nothing yet about arbitrary raw column comparisons. In particular it does not use lexicographic order on variable-length label vectors, which would be unsuitable for this conclusion.

9.2 Descendants of one graph are linearly ordered by reachability

Let $D(G)$ include G and its finite descendants, omitting the conventional self-loops $0[n] = 0$ from paths. Induct on the actual ordinal $\mu(G)$ to show that any two elements of $D(G)$ are comparable by reachability.

If one is G , this is immediate. Otherwise their paths begin at $G[i]$ and $G[j]$. Let $N = \max(i, j)$. By (2.3), both first-step graphs are prefixes of $G[N]$; repeated index-0 operations delete its excess last columns and reach either prefix. Thus both chosen descendants belong to $D(G[N])$. Equation (9.1) permits the induction hypothesis there.

Every nontrivial reachability path strictly lowers column order by (2.4). Therefore, within $D(G)$, column order agrees with reverse proper reachability: if $A <_{\text{col}} B$, then A is a proper descendant of B , and conversely. In particular $\mu(A) < \mu(B)$ whenever $A <_{\text{col}} B$ in this cone.

9.3 Combining all seeds, without a union-of-well-orders shortcut

For $m \leq n$, S_m is a prefix of S_n and is reached by index-0 deletions. If A descends from S_m and B from S_n , both descend from $S_{\max(m,n)}$. Section 9.2 therefore applies to every pair of finite standard graphs and gives

$$A, B \in \text{Std}, \quad A <_{\text{col}} B \implies \mu(A) < \mu(B). \tag{9.2}$$

The finite standard domain is an actual set: membership is witnessed by a natural seed index and a finite computation path. For any nonempty subset of it, the image under the actual function μ has a least ordinal. Any graph attaining that value is column-least by (9.2) and totality. This proves the standard column well-order without dependent choice, an infinite-branch argument, or the false claim that an arbitrary union of well-orders is well-ordered. Assigning rank κ to Top gives the optional one-point extension.

10. Remove $V = L$ by transferring the actual rank

Use the inner- L construction justified in IPD section 9. In a model of S , its constructible inner class has the same ordinals and natural numbers and satisfies $KP_\omega + V = L$. An ambient uncountable ordinal remains uncountable there, since an L -surjection would also be an ambient set function.

Run the argument in L . Its resulting μ and its ordinal target are actual sets in the ambient universe. Finite SPD syntax, normalization, comparison, and fundamental-sequence computations are absolute; so are finite paths from seeds. Every ambient raw step is the same finite step considered in L , and its strict ordinal rank inequality is absolute. The ambient universe can therefore minimize this transferred rank on any of its nonempty standard subsets, including subsets not in L .

The rank target may be countable in the ambient universe; that does not affect its being an actual ordinal. What transfers is the decreasing set function, not merely L 's assertion that it has no descending sequence. Thus the theorem holds in S itself.

11. Dependencies, proof status, and remaining formalization

Part	Justification	Not assumed
Finite heads and profiles	Finite recursion; IPD Appendix A actual LPO ranks; collection of finite layers	A global CAP or target SPD well-order
R^+	Bounded recursion on $(b, \rho(T), a)$; IPD section 4.3 history construction	FR/NP or supply as extra axioms
Closed heights	IPD sections 4.1 and 12.1-12.2, with the explicitly defined LastExt family	Regularity or ambient choice
EA and CS	Sections 6.1-6.2, including strict lower-profile induction for NP	Cross-parent weakening
Actual block	Canonical guard, typed carry, four calls, and pure-source identity	Arbitrary merge or unchanged new-seam heads
Standard order	Least final label, descendant-cone induction, common larger seed	Well-ordering of all raw column lists
Removal of $V = L$	Transfer of an actual rank, as in IPD section 9	Absoluteness of the mere absence of descending sequences

This is a paper-level completion of the fixed-rule well-ordering argument, conditional only on the explicit KP lemmas proved in the linked IPD manuscript and the standard KP inner- L interface stated there. It is not a new machine-checked theorem. In particular, existing IPD Lean results do not certify the changes to heads, LATENT, NP, carry, or connectors in this paper.

Formalization still needs the SPD finite syntax/implementation correspondence, recursive-head rank instantiation, doubled endpoint alphabet, the bounded R^+ recursion, LastExt closure, NP endpoint agreement and supply, exact four-call block lemma, and the final raw-rank-to-standard-order argument. Formalizing these in ordinary Lean and internally formalizing their stated KP bound are distinct tasks. Finite tests are useful implementation checks but discharge neither task.